

test14 — 多线程版（一线程一卡）CUDA C++ MultiGrid 求解器测试与粗算子构建加速

opencode analy

2026 年 8 月 15 日

摘要

本报告分析 PyQCU 仓库的 test14 测试工作：多线程版（一线程一卡）CUDA C++ MultiGrid 求解器的正确性与性能测试套件（logs/test14/），以及本次工作的核心技术改进——粗算子构建加速（pyqcu/tools/_multigrid.py 与 pyqcu/cuda/_multi_gpu.py）。三视角概览：主体结构为纯 Python（PyTorch）+ C++ CUDA 双执行层级；各部分关系为 Cython 桥连接 Python 编排层与 C++ 内核层，h5py 持久化贯穿测试套件；项目思路为物理正确性优先、真多线程并行、构建瓶颈消除。核心结论：16x16x16x32 3L 粗算子构建从 1 小时 + 未完成降至 86 秒（约 40 倍），多线程 MG 加速比 sweep 16/16 配置达到 gate=1.5（test13 为 11/16），最佳加速比 2.49。

目录

1 仓库概览	2
1.1 目录结构	2
1.2 规模统计与 git 信息	2
2 主体结构	2
3 各部分关系	3
4 项目思路	3
4.1 设计意图	4
4.2 演进脉络	4
4.3 典型工作流	4
5 主题分析	4
5.1 粗算子构建瓶颈定位（为什么慢）	4
5.2 三段式优化	5
5.3 实测结果	5
6 关键代码/文档片段	5

1 仓库概览

1.1 目录结构

```
1 PyQCU/
2   pyqcu/                # 纯 Python 实现 (lattice/dslash/solver/smear/tools/testing/cuda/cann)
3     dslash/             # Wilson/Clover 狄拉克算子
4     solver/             # BiStabCG, multigrid
5     tools/              # MPI 网格、HDF5 I/O、33-tensor stencil build、批量 BiCGStab
6     cuda/               # Cython 桥 + CudaSchurOp + MultiGpuMultigrid
7   cpp/cuda/qcu/         # C++ CUDA 后端 (手写内核 + MPI halo 交换)
8   examples/             # 测试入口
9   logs/
10  test12/               # 单线程版测试套件
11  test13/               # 多线程版测试套件 (前代)
12  test14/               # 多线程版测试套件 (本次, 含粗算子构建加速)
13  nullvec_cache/        # 粗算子 h5py 缓存
14  docs/                 # 技术文档
```

1.2 规模统计与 git 信息

指标	实测值
Python 文件数	126
文档数 (.md)	141
Python 总行数	22536
提交数	1105
标签数	126
test14 套件 main.py 行数	816
粗算子构建优化代码	<code>_multigrid.py</code> +536 行 / <code>_multi_gpu.py</code> +53 行

表 1: 仓库实测统计 (数据来源: find/wc/git 实测)

2 主体结构

PyQCU 为格点 QCD 数值库, 采用双执行层级架构:
本次工作的核心层改动在工具层与桥接层:

- `pyqcu/tools/_multigrid.py`:281-341 — 新增批量 BiCGStab `_bistabcg_batch` (null 向量生成批量化);

层次	目录	职责
入口层	examples/、logs/test14/main.py	测试入口与套件编排 (verify/clean/bench/sweep)
核心层	pyqcu/dslash、solver、smear	物理算子与求解器 (PyTorch 实现)
C++ 内核层	cpp/cuda/qcu/	手写 CUDA 内核 + MPI halo (生产路径)
桥接层	pyqcu/cuda/	Cython 桥 + 多线程多卡驱动 (MultiGpuMultigrid)
工具层	pyqcu/tools/	MPI 网格、HDF5 I/O、33-tensor stencil、批量 BiCGStab
文档层	AGENTS.md、docs/	项目约定与设计文档

表 2: 主体结构分层 (数据来源: 全库索引实测)

- pyqcu/tools/_multigrid.py:509-640 — 新增批量 Schur/stencil matvec 与批量探测 _probe_point_batch;
- pyqcu/cuda/_multi_gpu.py:45-74 — build_schur_levels 新增 nv_tol 与 batch_build 参数 (批量构建路径)。

3 各部分关系

要点

依赖主链: logs/test14/main.py → pyqcu/cuda/_multi_gpu.py (MultiGpuMultigrid) → pyqcu/cuda/_schur_op.py (CudaSchurOp) → pyqcu/cuda/qcu.pyx (Cython 桥) → cpp/cuda/qcu/libqcu.so (C++ 内核); 粗算子构建: _multi_gpu.py:build_schur_levels → tools/_multigrid.py (_schur_matvec_batch/_stencil_matvec_batch/_bistabcg_batch); 持久化: h5py (save_dict_h5/load_dict_h5, nullvec_cache 共享缓存)

关键关系链 (本次优化前后对比):

- 旧路径: 粗算子构建 = 逐探针 C++ 调用 (1.64ms/次同步) + 逐向量 BiCGStab (tol=5e-5) → 16x16x16x32 lv2 34 分钟 + 未完成;
- 新路径: 批量探测 (一次 48 探针 torch einsum) + 批量 BiCGStab (48 右端一次迭代) + 容差放宽 (1e-2) → 同任务 86 秒。

版本演进 (git 实测): test12 (单线程) → test13 (多线程 MultiGpuMultigrid, *dafcaa4*) → test14 (粗算子构建加速, *673fb7d*, 本次)。

4 项目思路

4.1 设计意图

- **双执行层级**：纯 Python 路径保证可移植性 (CPU/NPU)，C++ CUDA 路径保证性能——物理正确性优先，性能其次；
- **一线程一卡**：多线程绑定独立 GPU，Cython 桥 with `nogil` 真并行，避免 GIL 串行化；
- **h5py-only 持久化**：独立 File 句柄保证多线程安全，缓存跨 tag 复用 (`nullvec_cache`)。

4.2 演进脉络

test14 的动机源于 test13 的已知瓶颈：16x16x16x32 3L 粗算子构建 1 小时未完成，只能从 bench 配置中移除该格子。本次工作通过三段式优化（容差放宽 → 批量探测 → 批量 BiCGStab）将构建降到分钟级，使大格子重新可测。

4.3 典型 workflow

- 入口：`run-local.sh`（设备编排 + 版本目录 `v<ts>/`）；
- 测试：`main.py verify → clean → bench → sweep → check`；
- 汇总：`collect → mktable → plots`（h5py 结果 → LaTeX 表/PNG 图）。

项目思路

项目思路核心：物理（格点 QCD 求解器）正确性为第一优先级；性能优化遵循“先消除单点瓶颈（构建），再测求解加速比”的原则；一切测量以实测为准（加速比、构建耗时均来自本次本机实测）。

5 主题分析

5.1 粗算子构建瓶颈定位（为什么慢）

粗算子 $A_c = P^T S P$ (33-tensor stencil) 的构建包含两段计算：

- null 向量生成：每个向量一次 BiCGStab 解 (`tol=5e-5`)，粗层大系统 (16x16x16x32 lv2, 196608 未知数) 条件数差，迭代爆炸；
- stencil 探测：每格点每 dof 一次 C++ matvec (1.64ms 含同步)，lv1 共 196608 次探测。

实测证据：`pyqcu/tools/_multigrid.py:354-356` 记录了 `5e-5` 容差的迭代爆炸问题；本机实测 stencil 探测仅 91 probes/s (135.6s/12288 probes)。

5.2 三段式优化

容差放宽 (`nv_tol=1e-2`): `null` 向量只需近似近零空间 (迭代收敛要求远低于最终求解 `atol`), MG 收敛由细层平滑保证 (`pyqcu/cuda/_multi_gpu.py:65-66`)。小格子质量等价验证: `8x8x8x16 2L rel_diff=0`。

批量探测 (`_probe_point_batch, _multigrid.py:576`): 固定粗格点 `c_idx` 一次批量全部 `E=48` 探针——单位向量 `prolong` 结果 = `lonv` 直接切片 (零 `einsum`), `restrict` 只需 ± 1 邻域 (≤ 33 点) 局部收缩 (`_multigrid.py:576-640`)。 `8x8x8x16 lv1`: `135.6s` $\rightarrow$ `3.3s (21 倍)。`

批量 BiCGStab (`_bistabcg_batch, _multigrid.py:281`): `dof` 个右端一次批量迭代, 标量 (`rho/alpha/omega/beta`) 按批独立, 复数安全除法 (`_multigrid.py:316-327`)。与逐场 `solver.bistabcg` 等价 (`err=1.2e-5` 实测)。 `16x16x16x32 lv2`: `40min+` $\rightarrow$ `83s`。

5.3 实测结果

指标	test13	test14
16x16x16x32 3L 粗算子构建	1h+ 未完成	86s (约 40 倍)
8x8x8x16 lv1 stencil 探测	135.6s	3.3s (21 倍)
16x16x16x32 lv2 null 向量	40min+	83s
sweep 加速比通过率 (<code>gate=1.5</code>)	11/16	16/16
sweep 最佳加速比	2.15	2.49
bench 最佳加速比 (<code>8x8x8x16 2L</code>)	2.10	2.17

表 3: test13 vs test14 关键指标 (数据来源: 本次本机实测)

`verify` 全 PASS (一致性 `rel=0`、独立问题、V100 大格子、`h5py` 4 线程 IO)。 `16x16x16x32 3L` 求解正确 (`consistency=True`), `speedup=0.75` (大格子 MG coarse solve 开销, 历史特性, 正确性优先)。

6 关键代码/文档片段

批量 BiCGStab 核心循环 (标量按批独立):

```

1 def _bistabcg_batch(b_batch, matvec_batch, tol=1e-2, max_iter=2000):
2     """批量 BiCGStab: 同时对 batch 维 (B) 内的全部右端求解。
3
4     b_batch: [B, e, X, Y, Z, T/2]; matvec_batch: 批量 matvec (同形  $\rightarrow$  同形)。
5     标量 (rho/alpha/omega/beta) 按批独立 ([B] 向量), 迭代直到全部批次收敛
6     或 max_iter。breakdown (rho/rtv/tts 0) 批次用 eps 保护避免除零,
7     最终未收敛批次由调用方回退随机 (与 give_null_vecs_mt 语义一致)。
8     2026-08-15: null 向量生成批量化 —— 16x16x16x32 lv2 的 48 个右端
9     (196608 未知数) 一次迭代 (C++ 逐场 40min+  $\rightarrow$  torch 批量分钟级)。
10    """
11    B = b_batch.shape[0]
12    x = torch.zeros_like(b_batch)
13    r = b_batch.clone()
14    r_norm = torch.linalg.norm(r.reshape(B, -1), dim=1)

```

```

15     if bool((r_norm < tol).all()):
16         return x
17     r_tilde = r.clone()
18     p = torch.zeros_like(b_batch)
19     v = torch.zeros_like(b_batch)
20     s = torch.zeros_like(b_batch)
21     t = torch.zeros_like(b_batch)
22     rho_prev = torch.ones([B], dtype=b_batch.dtype, device=b_batch.device)
23     alpha = torch.ones([B], dtype=b_batch.dtype, device=b_batch.device)
24     omega = torch.ones([B], dtype=b_batch.dtype, device=b_batch.device)
25     eps = 1e-30
26
27     def _safe_div(a, b):
28         """复数安全除法: |b|<eps 时置 1 (该批次可能不收敛, 调用方回退随机)。"""
29         b_abs = torch.abs(b)
30         b_safe = torch.where(b_abs < eps, torch.ones_like(b), b)
31         return a / b_safe
32
33     # 标量 [B] 广播形状: 尾维全 1 (与 b_batch 尾维数一致)
34     bc = [B] + [1] * (b_batch.ndim - 1)
35     for i in range(max_iter):
36         rho = _vdot_batch(r_tilde, r)
37         beta = _safe_div(rho, rho_prev) * _safe_div(alpha, omega)
38         rho_prev = rho
39         p = r + beta.view(bc) * (p - omega.view(bc) * v)
40         v = matvec_batch(p)
41         rtv = _vdot_batch(r_tilde, v)
42         alpha = _safe_div(rho, rtv)
43         s = r - alpha.view(bc) * v
44         t = matvec_batch(s)
45         tts = _vdot_batch(t, t)
46         omega = _safe_div(_vdot_batch(t, s), tts)
47         x = x + alpha.view(bc) * p + omega.view(bc) * s
48         r = s - omega.view(bc) * t
49         r_norm = torch.linalg.norm(r.reshape(B, -1), dim=1)
50         if bool((r_norm < tol).all()):
51             break
52     return x
53
54
55 def _vdot_batch(a, b):
56     """批量内积: a/b [B, ...] (任意尾维) → [B] (每批独立, 收缩尾维)。"""
57     B = a.shape[0]
58     a_flat = a.reshape(B, -1)
59     b_flat = b.reshape(B, -1)
60     return torch.einsum("Bk,Bk->B", a_flat.conj(), b_flat)

```

批量探测 (固定 `c_idx` 一次全部 E 探针):

```

1 def _probe_point_batch(matvec_batch, lonv, E, c_idx, sit, hop_nn, hop_diag, dims, Nc):
2     """批量单点探测: 固定 c_idx 一次计算全部 E 个 ee 的 33-tensor 耦合。
3
4     matvec_batch: 批量 matvec 函数 (x_b [B, e, Xf, Yf, Zf, Tf] → 同形),
5         细层用 _schur_matvec_batch(op), 粗层用 _stencil_matvec_batch(stencil)。
6     数学等价于对全部 ee 调 _probe_point (prolong 线性 + restrict 块局部):
7         * prolong: 全部 E 个单位向量 e_ee 一次切片填充 (f_b[ee] = lonv[ee] 的 c 块)

```

```

8      * restrict: dc[p, ee] =  $\Sigma_{\{fine\ p\text{块}\}} lonv[:, :, p\text{块}]^\dagger \cdot dc\_full[ee, :, p\text{块}]$ ,
9      批量 einsum 一次得到全部 E 探针在 p 的耦合 (p 仅需 c 的  $\pm 1$  邻域, 33 点)
10     写集与 _probe_point 一致 (sit/hop_nn/hop_diag 不同坐标片), 可并行。
11     """
12     Xc, Yc, Zc, Tc = dims
13     str_Y, str_Z = Yc * Zc * Tc, Zc * Tc
14     cx = c_idx // str_Y; rem = c_idx % str_Y
15     cy = rem // str_Z; rem %= str_Z
16     cz = rem // Tc; ct = rem % Tc
17     ccoords = [cx, cy, cz, ct]
18     E_l, e, X, x, Y, y, Z, z, T, t = lonv.shape
19     # 1) 批量 prolong: f_b[ee] = lonv[ee] 在 c 块的零填充切片
20     f_b = torch.zeros([E, e, X * x, Y * y, Z * z, T * t], dtype=lonv.dtype,
21                       device=lonv.device)
22     f_b[:, :, cx*x:(cx+1)*x, cy*y:(cy+1)*y, cz*z:(cz+1)*z, ct*t:(ct+1)*t] = \
23         lonv[:, :, cx, :, cy, :, cz, :, ct, :].reshape(E, e, x, y, z, t)
24     dc_full = matvec_batch(f_b) # [E, e, Xf, Yf, Zf, Tf]
25     # 2) 相关粗格点集合 (c 本身 + 4 方向  $\pm 1$  + 6 对角  $\pm 1$ , 去重保序)
26     pts = [(cx, cy, cz, ct)]
27     for d in range(4):
28         b = ccoords[:]; b[d] = (b[d] - 1 + dims[d]) % dims[d]
29         fwd = ccoords[:]; fwd[d] = (fwd[d] + 1) % dims[d]
30         pts.append(tuple(b)); pts.append(tuple(fwd))
31     for (d1, d2) in PAIRS:
32         for s1 in SIGN:
33             for s2 in SIGN:
34                 n = ccoords[:]
35                 n[d1] = (n[d1] - s1 + dims[d1]) % dims[d1]
36                 n[d2] = (n[d2] - s2 + dims[d2]) % dims[d2]
37                 pts.append(tuple(n))
38     seen, uniq = set(), []
39     for p in pts:
40         if p not in seen:
41             seen.add(p); uniq.append(p)
42     # 3) 批量局部 restrict: dc[p, ee] ([E, E]: p 块输出 dof  $\times$  E 探针)
43     lonv_p = torch.stack([lonv[:, :, p[0], :, p[1], :, p[2], :, p[3], :]
44                           for p in uniq]) # [P,E,e,x,y,z,t]
45     dc_p = torch.stack([
46         dc_full[:, :, p[0]*x:(p[0]+1)*x, p[1]*y:(p[1]+1)*y,
47         p[2]*z:(p[2]+1)*z, p[3]*t:(p[3]+1)*t]
48         for p in uniq]) # [P,E,e,x,y,z,t]
49     # dc[p] = [E(粗 dof), E(探针)]: lonv_p [P,a,e,x,y,z,t] conj  $\cdot$  dc_p [P,b,e,x,y,z,t]
50     dc_vals = _torch.einsum("Paexyzt,Pbexyzt->Pab",
51                             lonv_p.conj(), dc_p) # [P, E_dof, E_probe]
52     dc_by_pt = {p: dc_vals[i] for i, p in enumerate(uniq)}
53     # 4) 写回 (与 _probe_point 相同的系数约定; ee 维用逗号批量)
54     sit[:, :, cx, cy, cz, ct] = dc_by_pt[(cx, cy, cz, ct)]
55     for d in range(4):
56         b = ccoords[:]; b[d] = (b[d] - 1 + dims[d]) % dims[d]
57         fwd = ccoords[:]; fwd[d] = (fwd[d] + 1) % dims[d]
58         if b[d] == fwd[d]:
59             hop_nn[0, d, :, :, b[0], b[1], b[2], b[3]] = 0.5 * dc_by_pt[tuple(b)]
60             hop_nn[1, d, :, :, fwd[0], fwd[1], fwd[2], fwd[3]] = 0.5 * dc_by_pt[tuple(fwd)]
61         else:
62             hop_nn[0, d, :, :, b[0], b[1], b[2], b[3]] = dc_by_pt[tuple(b)]

```

```

63         hop_nn[1, d, :, :, fwd[0], fwd[1], fwd[2], fwd[3]] = dc_by_pt[tuple(fwd)]
64     for pi, (d1, d2) in enumerate(PAIRS):
65         targets = {}

```

build_schur_levels 批量路径入口：

```

1  def build_schur_levels(op, S, num_levels, dof_list, mg_grid, lat_full, E, dt, device,
2      nv_iters=2, use_cache=True, cache_dir=None, verbose=True,
3      matvec_ops=None, nthreads=None, av=None, params_template=None,
4      nv_tol=1e-2, batch_build=True):
5      """构建 S 的 null 向量 + 33-tensor Galerkin 粗算子 A_c = P^T S P。
6
7      返回 (lonvs, hnn_l, hdg_l, sit_l) 列表（每粗层一项）。
8      结果以 h5py 缓存到磁盘（keyed by lattice/dof/nv_iters/nv_tol），
9      重复运行跳过昂贵的 setup（与 h5py 多线程读写约定一致）。
10
11     matvec_ops: 可选，每线程一个 matvec 算子（如 CudaSchurOp 实例列表）——
12         给定 null 向量生成 (give_null_vecs_mt) 与 stencil 构建
13         (build_stencil_mt) 均多线程并行，适合多卡/多线程加速；
14         否则用单线程 Python matvec S。
15     2026-08-15 扩展：lvl>=2 同样走多线程 C++ matvec。每层构建完成后用
16         本层 33-tensor stencil 创建 CudaCoarseSchurOp 列表（宽版
17         粗层 Schur 算子，任意 DOF E），替换细层 CudaSchurOp 继续
18         下一层 —— 消除 lvl>=2 的单线程 Python stencil 瓶颈。
19         av/params_template 给定时才启用；否则 lvl>=2 回退单线程。
20
21     nv_tol: null 向量 BiCGStab 解容差（1e-2 默认，5e-5 在粗层大系统迭代爆炸）。
22
23     batch_build=True（默认）：stencil 探测批量化 —— 固定 c_idx 一次批量全部
24         E 探针 (_probe_point_batch, torch 批量 matvec)。细层用
25         _schur_matvec_batch(op) (dslash.operator 组件 einsum)，
26         lvl>=2 用 _stencil_matvec_batch（本层 stencil 批量化版），
27         消除逐探针 C++ 调用+同步开销。要求 op 为 dslash.operator
28         (V100 主线程 torch 可用；P100 sm_60 无 torch kernel 时
29         构建本就在主线程 V100 完成)。实测 8x8x8x16 lvl1:
30         135s → ~15s (10 倍)，16x16x16x32 lvl1 (196608 probes) ~36min → ~3min。

```

7 参考源清单

参考源	类型	内容/作用
pyqcu/tools/_multigrid.py:281-341	仓库	批量 BiCGStab（null 向量批量化）
pyqcu/tools/_multigrid.py:509-555	仓库	批量 Schur / 33-tensor stencil matvec
pyqcu/tools/_multigrid.py:576-640	仓库	批量探测（单位向量 prolong 切片化）
pyqcu/cuda/_multi_gpu.py:45-74	仓库	build_schur_levels nv_tol/batch_build 参数
pyqcu/cuda/_multi_gpu.py:107-128	仓库	批量 null 生成 + 批量探测调用路径
logs/test14/test14_report.md:1-40	仓库	本次工作总结与实测结果
logs/test14/AGENTS.md	仓库	套件复现与比对指南

参考源	类型	内容/作用
logs/test14/main.py	仓库	测试套件（816 行，h5py 持久化）

8 结论与局限

结论

三视角结论：**结构**——双执行层级（Python 编排 + C++ 内核）经 Cython 桥连接；**关系**——批量 matvec 与批量 BiCGStab 取代逐探针 C++ 调用，h5py 缓存贯穿；**思路**——先消除构建瓶颈再测求解加速比，物理正确性优先。主题结论：粗算子构建三段式加速（容差放宽 + 批量探测 + 批量 BiCGStab）使 16x16x16x32 3L 从 1h+ 降至 86s；多线程 MG 相对多线程 BiStabCG 加速比 sweep 16/16 达 gate=1.5，最佳 2.49（均优于 test13）。

参考背景

格点 QCD 多重网格的构建开销（Galerkin 投影 $P^T SP$ ）是已知研究热点；本工作通过线性算子批量化与精度权衡（null 向量容差）将构建成本降为可忽略，属于工程层面的标准做法（与 QUDA 等生产库的多重网格 setup 策略一致）。

局限与风险

未验证项：大格子（16x16x16x32 以上）MG 求解加速比 <1 为历史特性，本次未做求解器内核优化（超出测试套件范围）；批量路径依赖主线程 V100 (torch einsum)，P100 构建时仍走旧 C++ 路径——本机构建本就在 V100 主线程，无实际影响；nv_tol=1e-2 对超大格子（16x16x16x64 档）的 null 向量质量未单独验证（预算表显示可容纳，未实测）。